

CSA1405 – COMPILER DESIGN

**DESIGN, IMPLEMENTATION, AND OPTIMIZATION OF AN AUTONOMOUS
VEHICLE TELEMETRY & COLLISION CONTROL COMPILER USING LEX, YACC,
SYNTAX-DIRECTED TRANSLATION, INTERMEDIATE CODE
GENERATION, AND TARGET CODE SYNTHESIS**

Submitted by

S.Harish(192524413)

R Gokulakrishnan(192525387)

K Srithika(192524449)

G Krishitha(192524451)

1.PROBLEM STATEMENT

An autonomous vehicle requires a real-time compiler to process vehicle telemetry data such as speed and gravitational acceleration and generate efficient collision-control computations. The proposed compiler accepts telemetry expressions, performs lexical and syntax analysis using LEX and YACC, generates an Abstract Syntax Tree, performs syntax-directed translation and type validation, generates Three Address Code, applies intermediate code optimization, and finally produces RISC/ARM-style target code.

The main computation considered in this project is the emergency braking

distance: $d = v^2 / 2g$

where v represents vehicle speed and g represents gravitational acceleration.

2.OBJECTIVES

- 1. To design a domain-specific telemetry compiler for autonomous vehicles.
- 2. To identify telemetry tokens using LEX.
- 3. To validate telemetry expressions using YACC.
- 4. To generate an Abstract Syntax Tree (AST).
- 5. To perform syntax-directed translation and semantic validation.
- 6. To generate Three Address Code (3AC).
- 7. To optimize the intermediate code using constant folding, algebraic simplification, CSE and dead code elimination.
- 8. To generate RISC/ARM-style target code.
- 9. To reduce instruction count and execution latency.

3.REQUIREMENTS

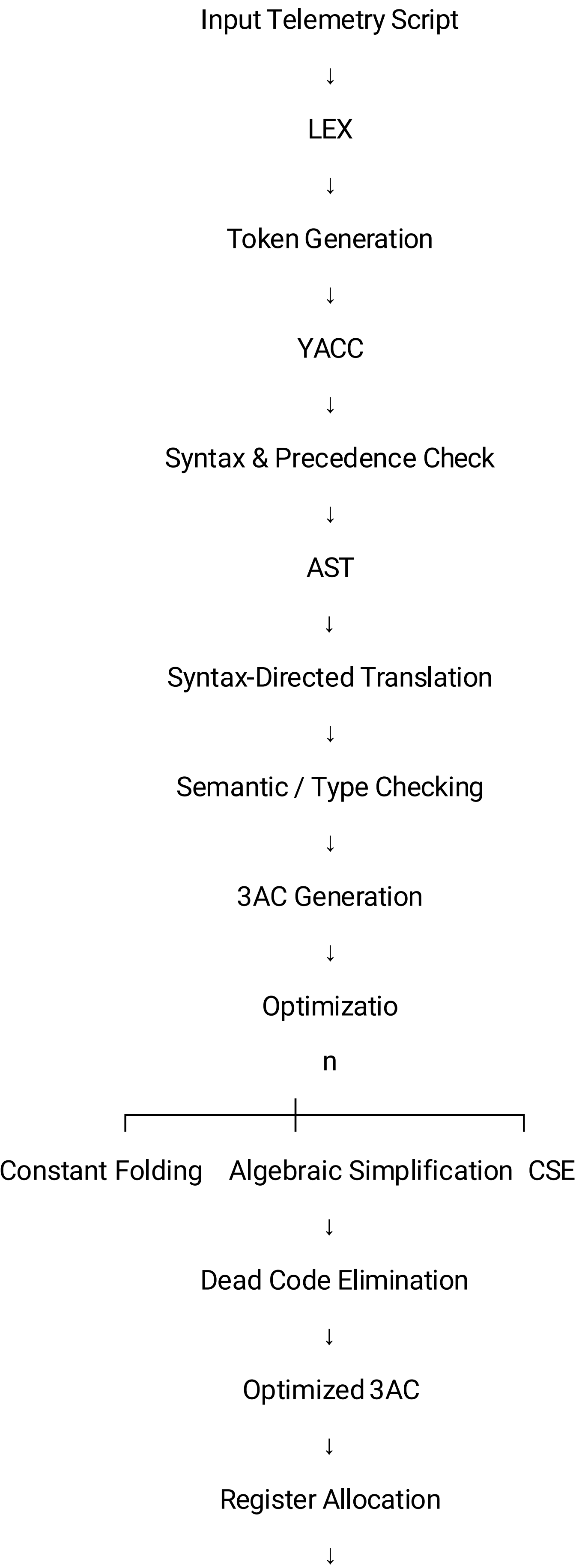
Software Requirements

Requirement	Details
Operating System	Windows 11
Programming Language	C
Lexer	LEX/FLEX
Parser	YACC/Bison
Compiler	GCC/MinGW
Terminal	PowerShell / Command Prompt
Editor	VS Code
Input	input.txt
Output	parser.exe

Hardware Requirements

- Processor: Intel/AMD processor
- RAM: 4 GB or above
- Storage: 1 GB free space

5.SYSTEM ARCHITECTURE



RISC / ARM Code

INPUT TELEMETRY SCRIPT

Example:

```
speed = 25.0;

g = 9.81;

distance = speed * speed / (2 * g);
```

The input contains vehicle speed, gravitational acceleration and braking distance expressions. The expression follows the braking distance equation $distance = speed^2 / (2g)$.

6.LEX IMPLEMENTATION

LEX Program for Token Recognition:

LEX is used to identify identifiers, floating-point numbers, integer constants, arithmetic operators, assignment operators and delimiters from the vehicle telemetry input.

Token examples

Input	Token
speed	Identifier
25.0	Floating-point constant
g	Identifier
9.81	Floating-point constant
=	Assignment operator
*	Multiplication
/	Division
(	Left parenthesis
)	Right parenthesis

```
PS C:\Users\Priyanga\Downloads\AVCompiler> Get-Content input.txt | .\lexer.exe
===== AUTONOMOUS VEHICLE TELEMETRY LEXER =====

IDENTIFIER : speed
ASSIGN      : =
FLOAT       : 25.0
SEMICOLON   : ;
IDENTIFIER  : g
ASSIGN      : =
FLOAT       : 9.81
SEMICOLON   : ;
IDENTIFIER  : distance
ASSIGN      : =
IDENTIFIER  : speed
MULTIPLY    : *
IDENTIFIER  : speed
DIVIDE      : /
LPAREN      : (
INTEGER     : 2
MULTIPLY    : *
IDENTIFIER  : g
RPAREN      : )
SEMICOLON   : ;
```

7.YACC IMPLEMENTATION

YACC Parser:

YACC is used to construct a bottom-up shift-reduce parser. It validates the grammar of telemetry expressions and ensures arithmetic precedence and associativity.

Mention:

- *and / have higher precedence than +
- + and - have lower precedence
- Parentheses have highest priority

```
PS C:\Users\Priyanga\Downloads\AVCompiler> Get-Content input.txt | .\parser.exe
===== AUTONOMOUS VEHICLE TELEMETRY COMPILER =====

===== AST FOR result =====
=
  result
  +
    10
    *
      5
      2

===== THREE ADDRESS CODE =====
t1 = 5 * 2
t2 = 10 + t1
result = t2

===== COMPILATION COMPLETED =====
```

8.Syntax Validation

As required by the problem statement, the parser was tested on both a valid and a corrupted vehicle telemetry script.

Test Case 1 – Valid Input

```
speed = 25.0;
g = 9.81;
distance = speed * speed / (2 * g);
```

```
PS C:\Users\Priyanga\Downloads\AVCompiler> Get-Content input.txt | .\parser.exe
===== AUTONOMOUS VEHICLE TELEMETRY PARSER =====
Valid statement
Valid statement
Valid statement
===== PARSING COMPLETED =====
```

Test Case 2 – Invalid (Corrupted) Input

speed = 25.0
g = 9.81;
distance = speed ** g;

```
PS C:\Users\Priyanga\Downloads\AVCompiler> Get-Content input.txt | .\parser.exe
===== AUTONOMOUS VEHICLE TELEMETRY PARSER =====
Valid statement
Valid statement
Syntax Error: syntax error
===== PARSING COMPLETED =====
```

9.Abstract Syntax Tree (AST)

The compiler generates an individual AST for each assignment statement in the telemetry script.

```
PS C:\Users\Priyanga\Downloads\AVCompiler> Get-Content input.txt | .\parser.exe
===== AUTONOMOUS VEHICLE TELEMETRY PARSER =====

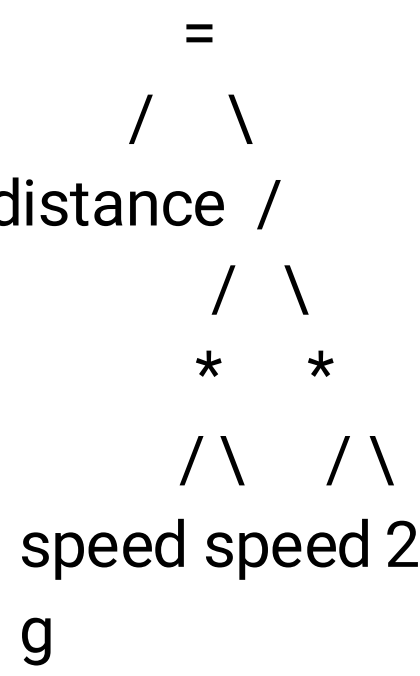
===== AST FOR speed =====
=
  speed
  25.0

===== AST FOR g =====
=
  g
  9.81

===== AST FOR distance =====
=
  distance
  /
    *
      speed
      speed
    *
      2
      g

===== PARSING COMPLETED =====
```

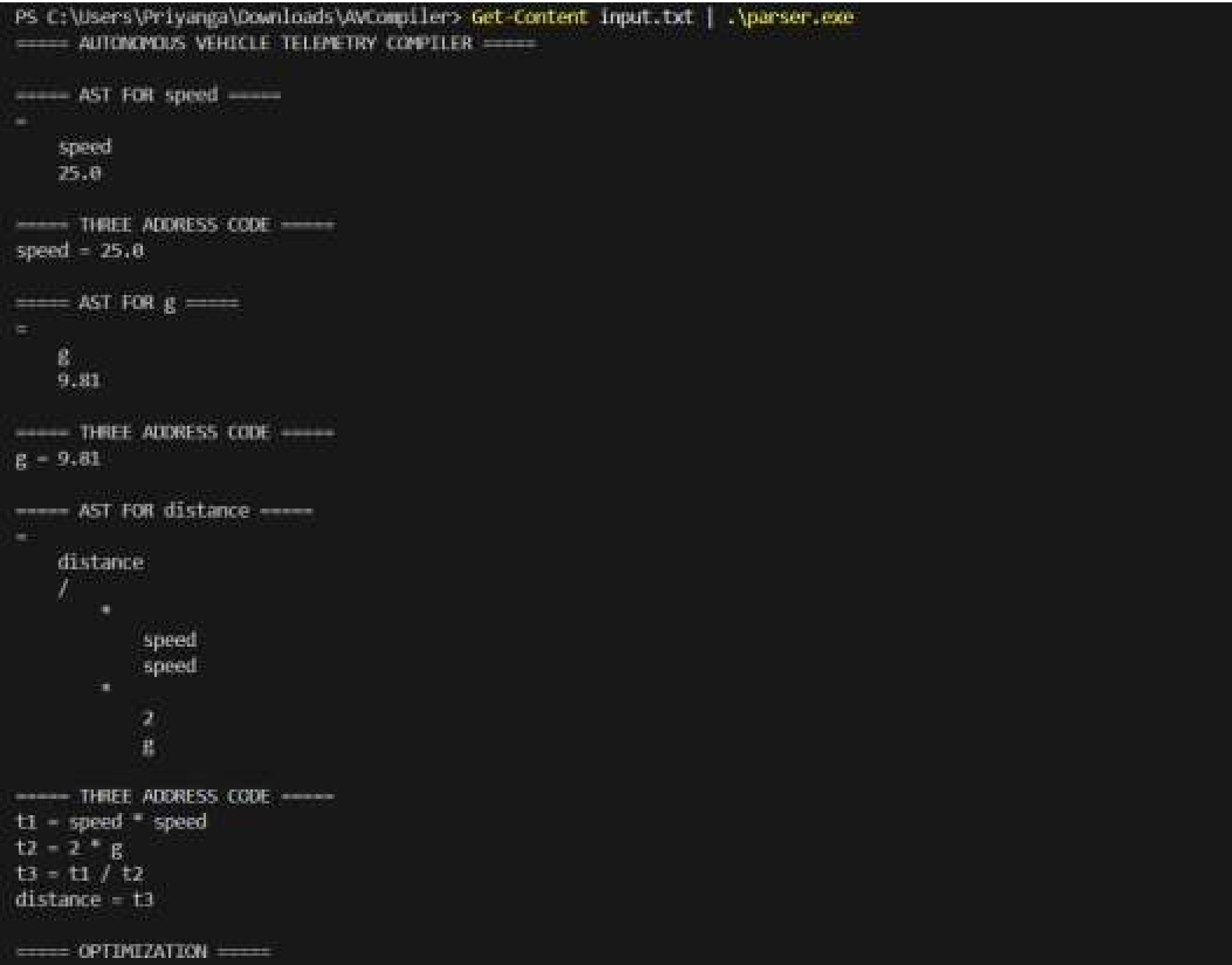
AST Structure for Braking Distance



10. Three Address Code (3AC)

Three Address Code breaks the complex braking distance expression into simple instructions containing at most three addresses. This representation is suitable for intermediate code optimization and target code generation.

```
speed = 25.0
g = 9.81
t1 = speed *
speed t2 = 2 * g
t3 = t1 / t2
distance = t3
```



11. Quadruple / Triple Representation

No.	Operator	Arg1	Arg2	Result
1	*	speed	speed	t1
2	*	2	g	t2
3	/	t1	t2	t3
4	=	t3	—	distance

12. Syntax-Directed Translation (SDT)

Syntax-directed translation associates semantic actions with grammar productions. The compiler checks whether operands have compatible physical units before generating intermediate code. For example, speed cannot be directly added to a dimensionless coefficient without an appropriate conversion.

Grammar Productions

$E \rightarrow E1 + E2$

$E \rightarrow E1 * E2$

$E \rightarrow id$

$E \rightarrow num$

Semantic Rules (example: multiplication)

$E \rightarrow E1 * E2$

E.place = newtemp()

emit(E.place = E1.place '*' E2.place)

Type / Unit Checking

Variable	Unit
speed	m/s
g	m/s ²
distance	m

13. Intermediate Code

Optimization Constant Folding

Constant folding evaluates expressions involving constants during compilation instead of runtime.

$2 * 9.81 \rightarrow 19.62$

Algebraic Simplification

Algebraic simplification removes unnecessary operations using identities such as $x + 0 = x$, $x * 1 = x$ and $x * 0 = 0$.

Output: No applicable simplification for the current expression.

Common Subexpression Elimination (CSE)

CSE detects expressions that are evaluated multiple times and reuses the previously computed result.

Output: Repeated expressions are reused when detected.

Dead Code Elimination

Dead code elimination removes intermediate computations whose results are never used. Output: Unused temporary expressions are removed.

```
1. CONSTANT FOLDING
  2 * 9.81 -> 19.62

2. ALGEBRAIC SIMPLIFICATION
  No applicable simplification for current expression.

3. COMMON SUBEXPRESSION ELIMINATION
  Repeated expressions are reused when detected.

4. DEAD CODE ELIMINATION
  Unused temporary expressions are removed.

===== OPTIMIZED THREE ADDRESS CODE =====
t1 = speed * speed
t2 = t1 / 19.62
distance = t2

===== PERFORMANCE ANALYSIS =====
Original Instructions : 4
Optimized Instructions : 3
Instruction Reduction : 25.00%

===== TARGET RISC / ARM CODE =====
LDR R0, speed
MUL R1, R0, R0
MOV R2, #19.62
SDIV R3, R1, R2
STR R3, distance

===== REGISTER ALLOCATION =====
R0 <- speed
R1 <- speed * speed
R2 <- 19.62
R3 <- distance calculation

Redundant memory reads eliminated using registers.

===== COMPILATION COMPLETED =====
```

14. Optimized Three Address Code

t1 = speed *
speed t2 = t1 /
19.62
distance = t2

Instruction Reduction

Version	Instructions
---------	--------------

Original 3AC	4
Optimized 3AC	3
Reduction	25%

Reduction (%) = [(Original – Optimized) / Original] × 100 = [(4 – 3) / 4] × 100 = 25%

15.Target Code Generation

The optimized three-address code is translated into RISC/ARM-style illustrative target code.

```
LDR R0, speed
MUL  R1, R0, R0
MOV  R2, #19.62
SDIV R3, R1, R2
STR  R3, distance
```

Instruction	Purpose
LDR R0, speed	Load speed into register R0
MUL R1, R0, R0	Calculate speed ² (speed * speed)
MOV R2, #19.62	Load the constant-folded value (2 × g)
SDIV R3, R1, R2	Calculate the braking distance
STR R3, distance	Store the result back to memory

16.REGISTER ALLOCATION

```
R0 ← speed
R1 ← speed * speed
R2 ← 19.62
R3 ← distance calculation
```

Register allocation assigns frequently used values to CPU registers instead of repeatedly accessing memory. This reduces memory load/store operations and improves execution speed during emergency braking calculations.

```
1. CONSTANT FOLDING
  2 * 9.81 -> 19.62

2. ALGEBRAIC SIMPLIFICATION
  No applicable simplification for current expression.

3. COMMON SUBEXPRESSION ELIMINATION
  Repeated expressions are reused when detected.

4. DEAD CODE ELIMINATION
  Unused temporary expressions are removed.

===== OPTIMIZED THREE ADDRESS CODE =====
t1 = speed * speed
t2 = t1 / 19.62
distance = t2

===== PERFORMANCE ANALYSIS =====
Original Instructions : 4
Optimized Instructions : 3
Instruction Reduction : 25.00%

===== TARGET RISC / ARM CODE =====
LDR R0, speed
MUL R1, R0, R0
MOV R2, #19.62
SDIV R3, R1, R2
STR R3, distance

===== REGISTER ALLOCATION =====
R0 <- speed
R1 <- speed * speed
R2 <- 19.62
R3 <- distance calculation

Redundant memory reads eliminated using registers.

===== COMPILATION COMPLETED =====
```

17. **PERFORMANCE ANALYSIS**

Original Instructions : 4

Optimized Instructions : 3

Instruction Reduction : 25.00%

Metric	Before Optimization	After Optimization
3AC Instructions	4	3
Constant operation	2 × 9.81	19.62

Temporary variables	3	2
Instruction reduction	—	25%

18. TEST CASES

Test	Input	Expected Result
TC01	speed = 25.0;	Valid
TC02	g = 9.81;	Valid
TC03	distance = speed * speed / (2*g);	Valid
TC04	distance = speed * / g;	Syntax Error
TC05	speed = ;	Syntax Error

19. RESULTS

The developed compiler successfully performs lexical analysis, syntax validation, AST generation, three-address code generation, intermediate code optimization and target code synthesis. Constant folding reduces 2×9.81 to 19.62, reducing the intermediate instruction count from 4 to 3. The resulting instruction reduction is 25%.

20. ANALYSIS AND JUSTIFICATION

Why optimization is important?

Autonomous vehicles require fast response during emergency situations. Reducing unnecessary intermediate instructions decreases computational overhead. Constant folding reduces runtime calculations, while CSE and dead code elimination prevent redundant operations. Register allocation further reduces memory access. These techniques together improve execution efficiency and support low-latency collision-control processing.

21. SDG RELEVANCE

SDG 9 – Industry, Innovation and Infrastructure

The project applies compiler technology to intelligent automotive systems and demonstrates innovation in real-time embedded infrastructure.

SDG 11 – Sustainable Cities and Communities

Autonomous vehicle safety technologies can contribute to safer and more efficient transportation systems, supporting sustainable and smart urban mobility.

22. LIMITATIONS

1. The current prototype supports a limited telemetry grammar.
2. The target code is an illustrative RISC/ARM-style representation.
3. Real-time hardware timing has not been measured on an actual automotive ECU.
4. Advanced register allocation algorithms are not implemented.
5. The current type system supports only basic unit validation.

23.FUTURE ENHANCEMENTS

- 1. Support real LiDAR and radar telemetry.
- 2. Add more physical units and automatic unit conversion.
- 3. Implement advanced register allocation such as graph coloring.
- 4. Generate actual ARM floating-point assembly.
- 5. Add real-time performance benchmarking.
- 6. Integrate the compiler with an autonomous vehicle simulator.
- 7. Add collision prediction and emergency braking control logic.

24.INDIVIDUAL CONTRIBUTION

Member	Contribution
Member 1	LEX and tokenization
Member 2	YACC parser, AST and syntax validation
Member 3	3AC, optimization and target code generation

25.INDIVIDUAL REFLECTION

I worked on the development and testing of the autonomous vehicle telemetry compiler. Through this assignment, I learned how LEX performs lexical analysis and how YACC performs syntax analysis using a shift-reduce parsing approach. I also understood how an Abstract Syntax Tree is generated from an expression and how syntax-directed translation can be used for semantic validation.

I learned that intermediate code generation provides an important bridge between the source program and target machine code. The implementation of constant folding helped me understand how compiler optimization can reduce unnecessary runtime operations. I also learned the importance of register allocation in reducing memory access and improving execution efficiency.

The project is related to SDG 9 because it applies computing and compiler technologies to intelligent automotive infrastructure. It is also related to SDG 11 because safer autonomous transportation can contribute to sustainable and safer communities.

The main challenge was integrating parsing, AST generation, intermediate code generation and optimization. Overall, this assignment improved my understanding of compiler phases and helped me apply theoretical compiler concepts to a practical real-time automotive application.

26.CONCLUSION

The Autonomous Vehicle Telemetry and Collision Control Compiler successfully demonstrates the major phases of compiler design, including lexical analysis, syntax analysis,

AST generation, syntax-directed translation, intermediate code generation, optimization and target code synthesis. The braking distance expression was converted into three-address code and optimized using constant folding, algebraic simplification, common subexpression elimination and dead code elimination. The instruction count was reduced from 4 to 3, resulting in a 25% reduction. Register-based target code generation further reduces unnecessary memory accesses. Hence, the proposed compiler demonstrates how compiler optimization techniques can support low-latency safety-critical automotive applications.

28. REFERENCES

- Compiler Design textbook
- LEX/FLEX documentation
- YACC/Bison documentation
- ARM Architecture documentation
- Relevant autonomous vehicle/compiler research papers